Software Architecture and Design Specification

Project: Basic ZIP Compression Tool Using Huffman Coding for File Size Reduction
Version: 1.0
Authors (Team 9):

RITESH P | PES2UG24CS340
PAVITHRA N | PES2UG24CS347
NIKSHEP V A | PES2UG24CS320
PRANAV BALEBAIL | PES2UG24CS362

Date: 03-10-2026
Status: Draft(Architecture Section Complete)

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Nikshep VA	Initial SAD draft with introduction, document overview, baseline component descriptions, and architectural data flow
1.1	03-10-2026	Pranav B	Revised Architecture section into IEEE format: updated component architecture, architectural pattern, technology stack, constraints, risks, SRS/STP requirement traceability and security architecture.
1.2	03-10-2026	Pavithra N	Design section with UML sequence diagrams ,API Design, Error handling , Logging and monitoring
1.3	03-10-2026	Ritesh P	UI/UX Design, Open Issues and Next steps, Appendices

Approvals

Role | name | Signature/Date

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Basic ZIP Compression Tool Using Huffman Coding for File Size Reduction (v1.0) in accordance with the standard IEEE Software Architecture and Design format.

1.2 Scope

Covers the end-to-end command-line compression and decompression workflow using byte frequency analysis (0–255 range), Huffman tree construction, prefix-free code generation, bit-level encoding/decoding, binary header creation and validation, file output, and cross-cutting resource management. Graphical user interfaces (GUI), multithreading, third-party compression algorithms (DEFLATE, LZ77, LZW), and network/cloud transfers are excluded.

1.3 Audience

Software developers, QA/test engineers, project instructors, and maintenance teams.

1.4 Definitions

Huffman Coding	A lossless entropy-encoding algorithm that assigns variable-length, prefix-free binary codes to input bytes based on their frequency of occurrence.
CLI (Command-Line Interface):	Text interface used to run compression/decompression commands.
Bitstream	The packed sequence of encoded bits forming the compressed file.
Prefix-Free Code	A code where no code is a prefix of another, allowing unambiguous decoding.
Binary Header	Metadata block storing format info needed to reconstruct the original file.
Compression Ratio	The ratio Original file size and compressed file size.
Lossless Compression	Compression that restores the exact original file with no data loss.

2. Document Overview

2.1 How to Use This Document

Provides the architectural and design specification for the Huffman-based file compression tool, including component structure, architectural pattern, requirement traceability, security architecture, UML sequence diagrams, API design, and error handling.

2.2 Related Documents

- **SRS v1.0:** Software Requirements Specification – Basic ZIP Compression Tool Using Huffman Coding (HZIP-F-001 to HZIP-F-020, HZIP-NF-001 to HZIP-NF-006, HZIP-SR-001 to HZIP-SR-006).
- **STP v1.0:** Software Test Plan – Basic ZIP Compression Tool Using Huffman Coding for File Size Reduction (TC-01 to TC-11 and RTM).

3. Architecture

3.1 Goals and constraints

This architecture is designed for:
Goals:

- Lossless byte-for-byte reconstruction
- correct Huffman coding
- reliable compression/decompression
- accurate size and compression ratio reporting
- controlled error handling
- safe resource management.

Constraints:
This architecture has the following constraints

- Command-line operation only
- basic Huffman coding
- standard C compiler environment (GCC/Clang)
- no GUI
- no multithreading
- no network/cloud transfer
- no third-party compression algorithms (DEFLATE, LZ77, LZW).

3.2 Stake holders and concerns

Stake holders	Concerns
End Users	Correct compression/decompression, visible compression ratio, overwrite safety, data integrity
Developers	Modularity, clean C interfaces, maintainability, ease of debugging
QA / Test Engineers	Testability, Valgrind memory verification, full requirement traceability (TC-01 to TC-11)
Project Instructors / Evaluators	IEEE-format documentation, complete SRS/STP requirement coverage, adherence to project constraints

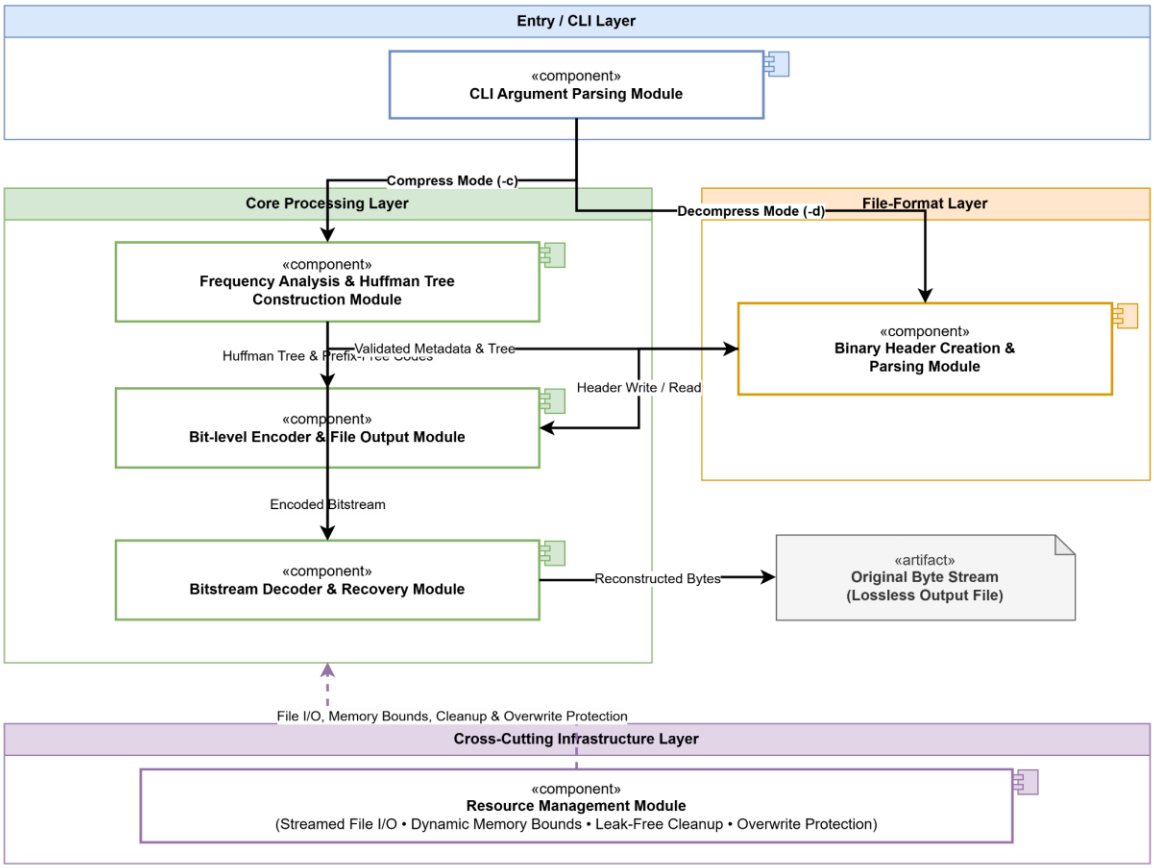
System / OS Environment	Compatibility with standard C compilers (GCC/Clang), no reliance on unsupported OS features

3.3Component Diagram and Description

Component (UML) Diagram

Figure 1. Component UML Diagram – Basic ZIP Compression Tool

- **Flow Overview:** CLI Argument Parsing → Frequency Analysis & Huffman Tree Construction → Bit-level Encoder & File Output ↔ Binary Header Creation & Parsing → Bitstream Decoder & Recovery
- **Cross-Cutting Support:** Resource Management ↔ file and memory handling across all core modules.



3.4 Component Descriptions

The system is decomposed into six primary components:

Component name	Description
CLI Argument Parsing Module	Validates command-line arguments and routes execution.

Frequency Analysis & Huffman Tree Construction Module	Analyzes byte frequencies and builds the Huffman tree and codes
Bit-level Encoder & File Output Module	Encodes data into bits and writes the compressed file
Binary Header Creation & Parsing Module	Creates and validates header metadata for encoding and decoding
Bitstream Decoder & Recovery Module	Decodes the bitstream and detects corrupted or truncated data
Resource Management Module	Handles file I/O, memory management, cleanup, and overwrite protection

3.5 Chosen Architectural Pattern and Rationale

Chosen architecture: **Modular Layered and Pipeline Architecture**

- **Entry Layer:** Represented by the CLI Argument Parsing Module, isolating command-line argument and path validation from core logic.
- **Core Processing Layer:** Contains Frequency Analysis & Huffman Tree Construction, Bit-level Encoding, and Bitstream Decoding & Recovery arranged as a sequential data transformation pipeline.
- **File-Format Layer:** Contains Binary Header Creation & Parsing and File Output operations.
- **Cross-Cutting Infrastructure Layer:** Resource Management provides controlled file I/O, bounded dynamic memory allocation, cleanup, and failure handling across all layers.

Rationale:
This model is chosen because, it provides

- **Standalone CLI tool** with clear, distinct processing stages — a layered design fits naturally
- **Separates** command-line handling, Huffman analysis, encoding/output, header processing, decoding/recovery, and resource management
- **Enables** isolated unit testing, easier maintenance, and clear requirement traceability
- **Meets project constraints:** standard C, no GUI, no multithreading, no network/cloud use, no third-party compression libraries

3.6 Technology Stack & Data Stores

- **Language:** Standard C
- **Compiler:** GCC or Clang.
- **Interface:** Command-Line Interface (CLI) on Linux / WSL2
- **Data Storage:** Local binary files using standard C buffered file I/O (stdio.h); no external database or cloud storage is used
- **File Format:** Structured binary header followed by the packed bitstream
- **Testing & Verification Tools:** Valgrind (memory leak analysis) and cmp / diff / sha256sum (byte-for-byte comparison)

- **Diagramming Tools:** draw.io and PlantUML

3.7 Risks & Mitigations

Architectural Risk	Impact	Mitigation
Inconsistent Huffman tree/code reconstruction	Decompressed output fails byte-for-byte verification	Keep tree construction rules and header metadata serialization deterministic across encoder and decoder.
Bit-level boundary and padding errors	Trailing bits in final byte produce extra bytes	Centralize bitstream operations and store the exact original byte length in the binary header.
Corrupted or truncated compressed files	Crashes or undefined decoding behavior	Validate header magic/version fields and detect premature end-of-file conditions during bitstream reads.
Memory leaks or excessive heap consumption	Degraded stability or out-of-memory failures on large files	Use stream-based buffered I/O, controlled allocation/deallocation, and verify with Valgrind.
Accidental output overwrite	Loss of existing user files	Validate output paths in CLI/Resource Management and enforce overwrite protection (HZIP-F-020).

3.8 Traceability to Requirements

The table below maps all functional, non-functional, and security requirements from the SRS v1.0 and STP v1.0 to their architectural components and test cases:

Requirement ID	**Architectural Component(s)**	**STP Test Case**	**Traceability / Architectural Responsibility**
HZIP-F-001 / F-002 / F-003	CLI Argument Parsing Module	TC-01	Validates compression/decompression operation modes, paths, and command-line options.
HZIP-F-004 / F-005	Frequency Analysis & Huffman Tree Construction Module	TC-02	Reads arbitrary binary files and calculates byte frequencies across the 0–255 range.
HZIP-F-006 / F-007	Frequency Analysis & Huffman Tree Construction Module	TC-03	Builds the Huffman tree and generates prefix-free binary codes.

Requirement ID	Architectural Component(s)	STP Test Case	Traceability / Architectural Responsibility
HZIP-F-008 / F-009 / F-010	Bit-level Encoder & File Output; Binary Header Creation & Parsing	TC-04	Encodes the input at bit level, writes required header metadata, and creates the compressed file.
HZIP-F-011 / F-012	Bit-level Encoder & File Output Module	TC-05	Calculates and reports original file size, compressed file size, and compression ratio.
HZIP-F-013 / F-014 / F-015	Binary Header Creation & Parsing; Bitstream Decoder & Recovery	TC-06	Validates/parses header metadata, reconstructs the Huffman tree, and performs lossless byte-for-byte decompression.
HZIP-F-016 / F-017	Frequency Analysis & Huffman Tree Construction; Bitstream Decoder & Recovery	TC-07	Handles empty (0-byte) files and single-symbol files consistently without crashes.
HZIP-F-018 / F-019 / F-020	CLI Argument Parsing; Resource Management Module	TC-08	Supports clear error reporting, memory/file cleanup, and output overwrite protection.
HZIP-NF-001	Bitstream Decoder & Recovery; Bit-level Encoder & File Output	TC-09	Guarantees lossless byte-for-byte reconstruction and binary correctness.
HZIP-NF-002	Resource Management Module	TC-09	Enforces stream-based processing to maintain dynamic memory bounds on large files.
HZIP-NF-003 / NF-005	All Core Processing Modules	TC-10	Ensures standard C compiler portability (GCC/Clang) across arbitrary binary inputs.
HZIP-NF-004 / NF-006	Resource Management Module	TC-10	Ensures controlled crash-free error handling and leak-free dynamic memory cleanup.

Requirement ID	Architectural Component(s)	STP Test Case	Traceability / Architectural Responsibility
HZIP-SR-001 / SR-002	Binary Header Creation & Parsing Module	TC-11	Validates magic bytes, format version numbers, and stored header metadata before decoding.
HZIP-SR-003 / SR-004	Bitstream Decoder & Recovery Module	TC-11	Detects corrupted compressed bitstreams, truncated files, and premature end-of-file conditions.
HZIP-SR-005 / SR-006	Resource Management; Bit-level Encoder & File Output	TC-11	Prevents raw data leakage in diagnostic output and safely handles file-I/O and memory allocation failures.

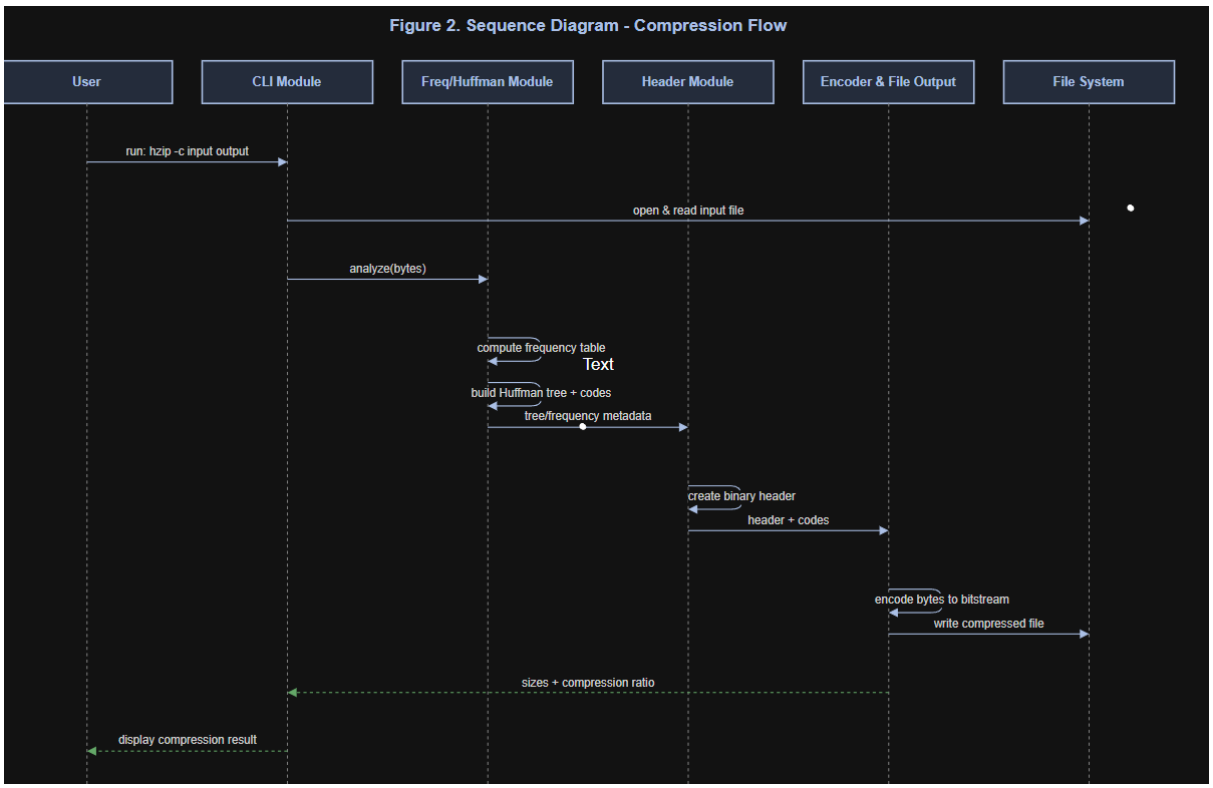
3.9 Security Architecture

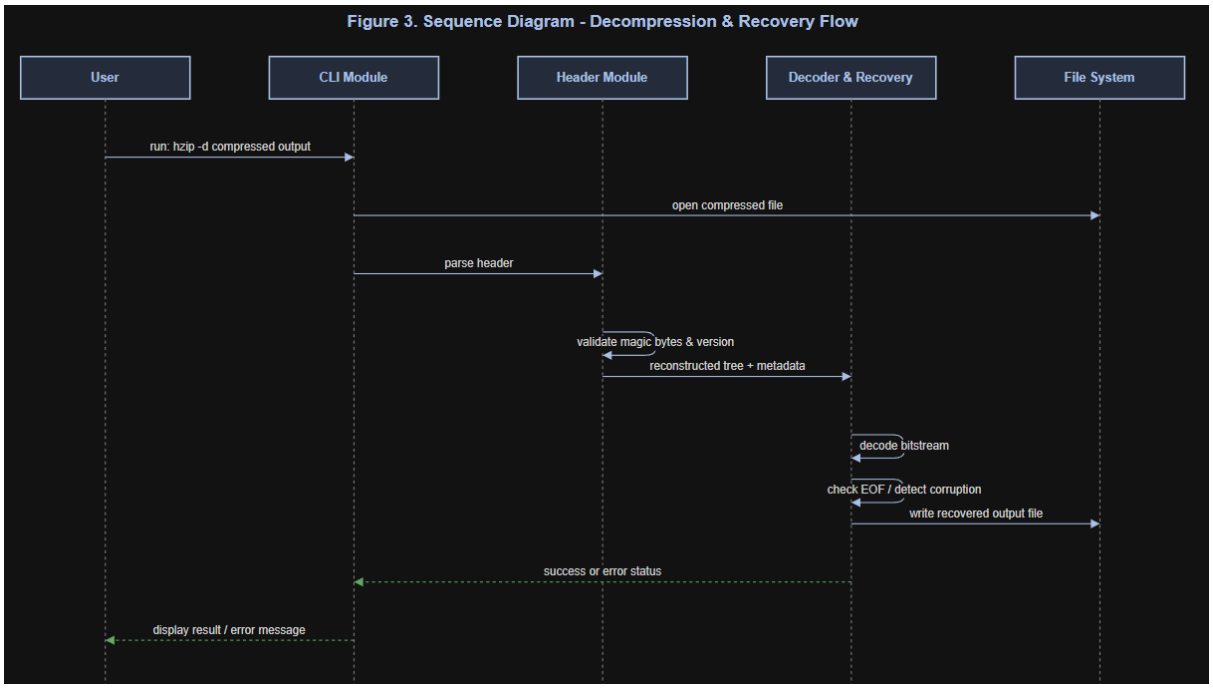
The security architecture focuses on data integrity, safe file handling, malformed-input rejection, and controlled resource use:

Security Domain	Requirement ID	Component(s)	Architectural Control
Header Validation	HZIP-SR-001, 002	Header Creation & Parsing	Validates magic bytes, version, and metadata; rejects invalid headers.
Corruption & Truncation Detection	HZIP-SR-003, 004	Bitstream Decoder & Recovery	Validates untrusted compressed data, tracks output size, and stops safely on corruption/EOF.
I/O & Memory Failure Handling	HZIP-SR-005, 006	Resource Management; Encoder & File Output	Checks file/memory allocation failures and provides sanitized errors.
Overwrite & Cleanup Protection	HZIP-F-019, 020; HZIP-NF-006	CLI Argument Parsing; Resource Management	Prevents unintended overwrites and ensures cleanup of files and memory on all exits.

4. Design

4.1 UML Sequence Diagrams





4.2 API Design

The interfaces below define the internal C function signatures for the two core components most central to correctness: the Frequency Analysis & Huffman Tree Construction module and the Binary Header Creation & Parsing module. Other modules (CLI, Encoder, Decoder, Resource Management) follow the same simple, return-code style.

4.2.1 Frequency Analysis & Huffman Tree Construction Module

Function Signature	Description
int compute_frequencies(const unsigned char *data, size_t length, unsigned int freq[256]);	Counts how often each byte appears. Returns 0 on success, -1 on bad input.
HuffmanNode *build_huffman_tree(const unsigned int freq[256]);	Builds the Huffman tree from those counts. Returns the root node, or NULL if empty.
int generate_codes(HuffmanNode *root, char *codes[256]);	Walks the tree to create a binary code for each byte. Returns 0 on success.
void free_huffman_tree(HuffmanNode *root);	Frees the tree's memory after use

4.2.2 Binary Header Creation & Parsing Module

Header file: fileformat.h

Function Signature	Description
int write_header(FILE *out, const Header *header);	Writes file metadata (magic bytes, version, size, frequency table) to the output. Returns 0 on success, -1 on error.
int read_header(FILE *in, Header *header);	Reads and checks the metadata before decoding. Returns 0 on success, -1 if missing or invalid (HZIP-SR-001, HZIP-SR-002).
int validate_header(const Header *header, size_t compressed_file_size);	Double-checks the header makes sense (valid size, non-empty table) before decoding begins. Returns 0 if valid, -1 if not.

4.3 Error Handling, Logging & Monitoring

4.3.1 Error Handling Strategy

1. **Return codes:** Every function returns 0 for success or a negative code for errors like file, memory, format, or argument issues — no failure goes unnoticed.

2. **Fail fast, clean up always:** On error, memory/files are released immediately using a single cleanup path, preventing leaks (HZIP-F-019, HZIP-NF-006).
3. **User-facing messages:** The CLI shows simple error messages (e.g. "Error: input file not found") without exposing raw data or memory addresses (HZIP-SR-004).
4. **Input validation boundary:** File data is treated as untrusted until validated; decoding starts only after validate_header() passes.

4.3.2 Logging

The tool doesn't have any log file, since it is a single-run CLI utility. Instead, it prints status information directly to the console:

- **Standard output:** Compression/decompression progress, original size, compressed size, compression ratio, and a final success message.
- **Standard error:** All error and warning messages, keeping them separate from normal program output so they can be redirected or filtered independently.

4.3.3 Monitoring

Formal runtime monitoring is out of scope for a standalone CLI tool. Correctness and resource behaviour are instead verified through testing and tooling defined in the STP:

- **Valgrind :** Used during development and testing to confirm there are no memory leaks or invalid accesses (TC-09, TC-10).
- **Exit codes:** Returns 0 on success, non-zero on failure, so scripts can detect issues automatically

4.4 Other Design Sections

4.41 Design Overview

- The system follows a layered, modular design with six components: CLI, Frequency/Huffman, Header, Encoder/File Output, Decoder/Recovery, and Resource Management
- Compression and decompression are two separate one-way flows that never call each other directly — they only interact through the compressed file's binary header
- Each module has a single responsibility and communicates through simple C function calls with return-code error handling, making the system easy to test and trace back to SRS requirements

4.42 UX/CLI Design

- The tool is operated entirely through command-line arguments — no GUI, consistent with project constraints
- Basic usage: hzip -c <input> <output> to compress, hzip -d <input> <output> to decompress

- On success, the CLI prints original size, compressed size, and compression ratio to standard output
- On failure, a short, plain-language error message is printed to standard error (e.g. Error: input file not found), and the program exits with a non-zero status
- If the output file already exists, the CLI prompts for confirmation before overwriting (HZIP-F-020)

4.4.3 Open Issues and Next Steps

- Current scope is limited to single-file compression; multi-file/folder archiving is a possible future extension
- Performance on very large files (500 MB+) has not yet been benchmarked and may need buffered/streaming improvements
- Further stress testing with randomly corrupted files is planned to harden the Decoder & Recovery module ahead of final submission

5. Appendices

5.1 Glossary

- **Huffman Coding** – Frequency-based variable-length encoding
- **CLI** – Command-line text interface
- **Bitstream** – Packed encoded bit sequence
- **Prefix-Free Code** – No code overlaps another
- **Binary Header** – Compressed file metadata block
- **Compression Ratio** – Original vs compressed size
- **Lossless Compression** – Exact, no data loss
- **SRS** – Requirements specification document
- **STP** – Test planning document
- **RTM** – Requirement-to-test mapping

5.2 References

- SRS, Team 9 – File Compression Tool, v1.0
- STP, Team 9 – File Compression Tool, v1.0
- Requirements Traceability Matrix (RTM), Team 9
- IEEE Std 1016-2009 — Recommended Practice for Software Design Descriptions

5.3 Tools

- **draw.io** – Used to create the UML sequence diagrams (Section 4.1)
- **GCC/Clang** – Standard C compiler used for building the project
- **Valgrind** – Memory-leak and invalid-access detection during testing
- **cmp / diff / sha256sum** – Binary and checksum comparison utilities for verifying byte-for-byte decompression accuracy